

# 流量治理：从大促 GMV 倒推的五层闸门

业务视角下的 TokenBucket / SlidingWindow / CircuitBreaker / 异步削峰 / HTTP cache

---

gomall · 业务侧 deck (v2)

# 今日大纲

业务困局：流量治理到底治什么

第一层：TokenBucket 一商家和爬虫的边界

第二层：SlidingWindow 一抢券公平与黑产 ROI

第三层：CircuitBreaker 一支付宕机不让交易链路死

第四层：异步下单一双 11 0 点不转圈

第五层：HTTP cache 一让读流量根本不到 gomall

客服话术：业务码就是产品

大促运营 SOP：T-7 / T-1 / T 日 / T+1

SLO 与业务边界

多层叠加 / 级联熔断 / 全链路压测

## 业务困局：流量治理到底治什么

---

## 一个真实的双 11 场景：5 秒转圈背后是多少 GMV

- 0 点开抢，预估峰值 **100k QPS** 涌进 `/orders/create`。
- 同步链路顶不住：DB 连接池打满 → 用户看到 **5 秒转圈**。
- 行业数据：每多 1 秒延迟，下单放弃率 + 7%；**5 秒  $\approx$  38% 放弃**。
- 算账：单平台 0 点 10 亿 GMV 预算  $\times$  38% = **3.8 亿损失**，比限流误伤几百用户严重几个数量级。
- 结论：流量治理不是“挡攻击”，是用业务可接受的代价换 **GMV 不雪崩**。

*stressTest/REPORT.md §1 基线 64k RPS；internal/order/async.go:1-187 异步削峰方案*

## 5 个角色，5 种” 流量治理是什么”

| 角色    | 体感               | 关心             |
|-------|------------------|----------------|
| C 端用户 | 看到” 系统繁忙”        | 我能不能再点 / 多久能买到 |
| 商家    | 秒杀页打不开           | 我的店今天损失多少订单    |
| 运营    | 0 点峰值能不能扛        | GMV 影响 / 黑产薅多少 |
| 客服    | 70001 / 70002 投诉 | 怎么回话 / 怎么转 SRE |
| SRE   | 上游 429 / 下游慢     | SLO 是否还在预算内    |

对用户是” 重试一下”、对客服是话术、对运营是今天又少了多少单、对 SRE 是 SLO 烧多少预算。

这份 deck 按这 5 个视角讲，不止讲算法。[pkg/e/code.go:57-58](#) 70001 / 70002; [README.md](#) 业务全景表

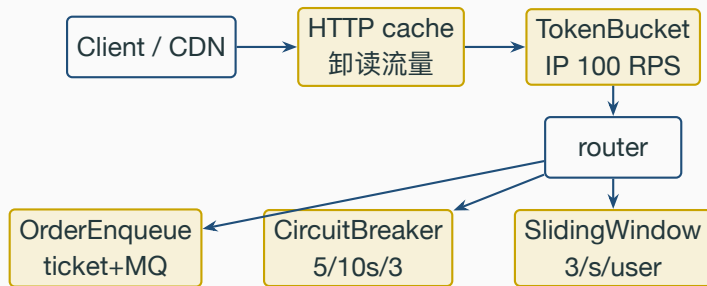
## 核心哲学：宁可下游慢、不能上游 429

- README 写明的 SLO 取舍：P0 核心交易（/orders/create / /paydown）**99.95%**（年宕 4h22min）、**不挂用户限流**。
- 业务翻译：**宁可让用户等 5 秒，也不能让用户看到 429**。
  - 等 5 秒：体验差，但订单还在；
  - 看到 429：用户立刻关 app，订单丢，怪平台不怪自己。
- P0 路径靠**异步削峰 + 缓存 + 库存预扣扛峰值**，限流只挂全局兜底（IP 100 RPS）。
- P2 营销路径反过来：宁可误杀真人、不能让黑产薅光（3/s/user 激进阈值）。
- 这两套取舍分别声明在各领域的 routes.go 里（router.go 只做组合根），是流量治理”按业务等级分挂载”的核心。

README.md SLO 表；routes/router.go:41 全局令牌桶；internal/payment/routes.go:14-21；

internal/skill/routes.go:17-24

## gomall 五层闸门的业务定位



五层不是流水线串联，是按接

□按业务分级挑选挂载：每层挡一个具体的角色痛点。*routes/router.go:41* 全局令牌桶；其余挂载点见 *internal/{product,redpacket,skill,payment,order}/routes.go*

## 五层闸门 vs 五个角色：哪层解决谁的痛

| 层              | 拦谁   | C 端体感        | 商家 / 运营关心    | 客服         | 每层都 |
|----------------|------|--------------|--------------|------------|-----|
| HTTP cache     | 重复读  | 详情秒开         | CDN 卸了 80% 读 | —          |     |
| TokenBucket    | 爬虫脚本 | 几乎无感         | GMV 保护       | —          |     |
| SlidingWindow  | 撞库脚本 | 慢一点没事        | 反黑产 / 抢券公平   | 70001      |     |
| CircuitBreaker | 慢依赖  | ”1 分钟后再试”    | 防雪崩 / 守 SLO  | 70002      |     |
| OrderEnqueue   | 写峰值  | 拿 ticket 等一会 | 双 11 不挂      | ”已入队、稍候查询” |     |

对应一个”角色 + 业务码 + 话术”，不是孤立算法。[pkg/e/code.go:57-58](#) 70001/70002；

[README.md](#) 5 角色业务全景表



## 核心 P0 vs 营销 P2：阈值取舍完全不同

| 等级    | 接口举例                      | 限流姿态       | 熔断  |
|-------|---------------------------|------------|-----|
| P0 交易 | /orders/create, /paydown  | 不挂用户限流     | 必须有 |
| P1 读  | /product/show, /carousels | HTTP cache | 不需要 |
| P2 秒杀 | /skill_product/skill      | 用户 3/s 激进  | 不需要 |
| P2 红包 | /redpacket/claim          | 用户 3/s     | 不需要 |
| P3 信息 | /category/list, 轮播图       | 全局 + CDN   | 不需要 |

**P0 宁可下游慢、不能**

**上游 429；P2 宁可误杀、不能让黑产薅光。**

两类接口用同一套阈值就两头不讨好：P0 损 GMV、P2 黑产薅干。  
*routes/router.go:41* 全局令牌桶；*internal/product/routes.go:15-16*；*internal/redpacket/routes.go:20-25*；*internal/payment/routes.go:14-21*；*internal/skill/routes.go:17-24*

## 第一层：TokenBucket —商家和爬虫 的边界

---

## 业务困局：爬虫脚本一晚上抓走多少商品数据

- 商家最讨厌的事：竞对爬虫夜里 5 QPS 持续 8 小时，把全店 SKU、价格、库存全抓走。
- 不拦的代价：
  - 数据被对家拿去比价杀价；
  - 单 IP 5 QPS 累计一夜 14 万次查询，DB 多扛 1 个商品详情连接池。
- 真用户单 IP 一晚上访问 /product/show 通常 < 50 次（浏览几十个商品）。
- TokenBucket **100 RPS / 200 burst** 的业务含义：
  - 真用户首屏并发 30+ 张图、瞬时 burst 100 → 200 桶吸收，**0 误伤**；
  - 爬虫稳态 5 QPS 不超限，但触发**风控 + 业务监控告警**，配合 IP 段封禁。
- 这层是**兜底闸门**，业务侧精细限流挂掉时至少不会被压死。

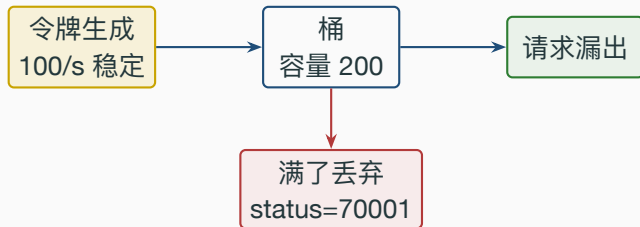
*routes/router.go:41 TokenBucket(rate.Limit(100), 200); middleware/ratelimit.go:23-50*

## 为什么 100 RPS / 200 burst: 基线 64k 反推

- gomall 单机基线 **64,254 RPS** (/ping, stressTest §1)。
- 真实用户单 IP < 1/600 单机容量; 100 RPS 留够 600× 安全余量。
- burst = 2× rps 的业务依据:
  - 浏览器一个详情页 30+ 静态 GET 是常态;
  - 商品详情图懒加载、瀑布流 → 瞬时 burst ≈ 100;
  - 200 桶让一次正常加载**绝不触发** (业务零误伤)。
- 商家视角: 阈值定太松爬虫拿数据、定太死客户投诉; 100/200 是用 **64k 基线 / 真实页面行为**双向算出的折中。

*routes/router.go:41; stressTest/REPORT.md §1 单机 64,254 RPS*

## TokenBucket 算法：桶 + 令牌生成 + 漏出



- 令牌按 rps 匀速生成；桶满则溢出。
- 请求拿到令牌才放行，拿不到立刻 70001（不阻塞、不排队）。
- burst 决定能否吸收**瞬时尖刺**（首屏并发 / 用户狂点）。
- 业务含义：**允许尖刺 / 拒绝高水位**——真用户偶尔狂点不痛、爬虫稳态 5 QPS 触不到这层，但配合 IP 黑名单可以拦。

*middleware/ratelimit.go:23-50 golang.org/x/time/rate 实现*

# TokenBucket 中间件实现 (逐行)

Listing 1: *middleware/ratelimit.go:23-50*

```
23 func TokenBucket(rps rate.Limit, burst int) gin.HandlerFunc {
24     var mu sync.Mutex
25     limiters := make(map[string]*rate.Limiter)
26     get := func(ip string) *rate.Limiter {
27         mu.Lock(); defer mu.Unlock()
28         if l, ok := limiters[ip]; ok { return l }
29         l := rate.NewLimiter(rps, burst); limiters[ip] = l; return l
30     }
31     return func(c *gin.Context) {
32         if !get(c.ClientIP()).Allow() {
33             c.JSON(200, gin.H{"status": e.ErrRateLimitExceeded})
34             c.Abort(); return
35         }
36         c.Next()
37     }
38 }
```

□ map 按 IP 隔离 (NAT 出口共享一个桶是已知代价); □ 每 IP 一个 `rate.Limiter`; □ `Allow()` 非阻塞, 不留排队队列; □ 拒了直接 200+70001 (不发 5xx, 详见熔断章节); □ 进程内 map 是单机代价, 多实例要换 Redis token bucket。

## 第二层：SlidingWindow —抢券公平 与黑产 ROI

---

## 业务困局：脚本党 10 万次刷券 = 真人少几张

- 秒杀 / 抢券场景：业务对手是脚本，不是用户。
  - 真人最快速度 300-500ms / 次 → 1s 最多 2-3 次；
  - 脚本不限速可以 1s 打 1000 次；
  - 撞库账号一夜  $24\text{h} \times 3600\text{s} \times 3 = 26$  万次抢券机会。
- 不拦的业务后果（算账）：
  - 10 万张优惠券，脚本 100 个号  $\times$  1 千次 = 100 万次抢 → 99% 命中率 = **9.9 万张** 被脚本抢走；
  - 真人只剩 **1%** (1000 张) → 用户体感“根本抢不到” → 大促口碑崩。
- SlidingWindow 3/s/user 把脚本打回客户端，反黑产 ROI 立等可现。

*internal/redpacket/routes.go:20-25 红包；internal/skill/routes.go:17-24 秒杀；internal/skill/service.go 秒杀逻辑*



## 阈值 3/s/user：用户 vs 脚本的精确分界

- 真用户行为画像：
  - 双击 + 网络重试，瞬时 2 次同秒是常态；
  - 留 50% 余量 = **3 次 / 秒上限**，绝不误伤。
- 脚本行为画像：第 4 次开始 70001，再脚本写多猛都白搭。
- TokenBucket 在此场景失效的原因：
  - NAT 出口（学校 / 公司 / 4G 基站）下 **所有用户共享一个 IP**；
  - 阈值放宽给真人 → 脚本一起放；阈值收紧拦脚本 → NAT 下真人被误伤。
- **必须按用户 ID 限，必须分布式精确**（多实例下 Redis 共享状态）。

*internal/skill/routes.go:17-24 Scope:seckill Limit:3 Window:1s ByUser:true*

## 压测实测：3/s/user 阈值 2.2% 误差

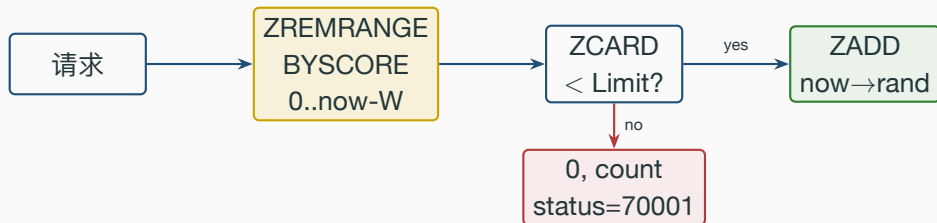
场景：30 VU  $\times$  15s 打 /skill\_product/skill，共用同一 token（复现同一用户脚本的请求形态）。

| 指标      | 实测          | 业务期望                |
|---------|-------------|---------------------|
| 通过请求数   | <b>46</b>   | $15s \times 3 = 45$ |
| 被限流数    | 781,624     | 黑产全拦                |
| 单接口 RPS | 52,082      | 与 ping 同量级          |
| 误差      | <b>2.2%</b> | $< 5\%$             |

业务翻译：30 个脚本号刷了 78 万次，真用户名额一张没少；服务端只做一次 Lua RTT (52k RPS 同 ping 量级)，脚本成本被推到客户端——这就是反黑产的 ROI。

`stressTest/REPORT.md §5`；`stressTest/seckill_rate_limit.js`

## SlidingWindow 在 Redis ZSet 上的实现



- 每个用户一个 ZSet: `rl:seckill:user:42`;
- `score` = 请求时间戳 (ms); `member` = "时间戳 + 6 字节随机" 避免同毫秒冲突;
- 滑动窗口 = 每次请求先踢掉 **window** 之外的旧记录, 再 ZCARD 数当前窗口内的请求数;
- 业务正确性: 没有"窗口边界双倍突刺", 反黑产精度高。

*repository/cache/ratelimit.go:18-34 Lua 脚本; middleware/ratelimit.go:70-106 调用方*

Listing 2: *repository/cache/ratelimit.go:18-34*

```
18 local key, window_ms = KEYS[1], tonumber(ARGV[1])
19 local limit, now_ms, member = tonumber(ARGV[2]), tonumber(ARGV[3]), ARGV[4]
20 redis.call('ZREMRANGEBYSCORE', key, 0, now_ms - window_ms)
21 local count = redis.call('ZCARD', key)
22 if count >= limit then return {0, count} end
23 redis.call('ZADD', key, now_ms, member)
24 redis.call('PEXPIRE', key, window_ms)
25 return {1, count + 1}
```

逐行：□ 入参 key/窗口/阈值/now/请求 id；□ **ZREMRANGEBYSCORE** 滑掉过期（业务正确性的关键）；□ ZCARD 数当前窗口；□ 超阈值返回 0；□ 通过则 ZADD 入；□ PEXPIRE 自动回收（不留垃圾 key）；□ 整脚本单原子，无竞态，是抢券公平的工程保证。

## 红包接口：同一套机制再用一次

- /redpacket/claim 也是 **Limit:3 Window:1s ByUser:true** (internal/redpacket/routes.go:20-25)。
- 与秒杀不同的是：红包额外挂**幂等**（避免一次抢包重复扣余额）。
- 业务模式归纳：
  - **SlidingWindow = P2 营销标配**（反黑产 / 抢券公平）；
  - **幂等 = P0/P2 资金链路标配**（防重复扣款）；
  - 两个中间件可以任意组合，业务侧一行声明搞定。
- 运营视角：双 11 / 春节 / 618 上新活动，复用现有中间件 = **零新代码、当天上线**。

*internal/redpacket/routes.go:20-25 红包抢包；internal/skill/routes.go:17-24 秒杀；幂等链*

*internal/order/routes.go:12, 14 与 internal/payment/routes.go:20, 26*

### 第三层：CircuitBreaker —支付宕机 不让交易链路死

---

## 业务困局：第三方支付 30s 超时 = 全站雪崩

- 真实事故剧本（行业公开过的案例）：
  - 13:42 第三方支付网关网络异常，RT 从 200ms 飙到 30s；
  - 13:43 /paydown 全员 30s 超时 → goroutine 堆积 → 内存爆 → **gomall 自身挂**；
  - 13:45 商家秒杀页打不开、订单详情打不开 → 客服炸锅；
  - 13:50 SRE 才看到告警——但人已经救不回来了，全站雪崩 40min。
- 不熔断的代价：**下游慢 = 自己跟着挂**（连锁雪崩比单点宕机损失大 10×）。
- CircuitBreaker 的业务含义：**我代下游回答你不行，免得你等 30 秒还白等。**
- P0 SLO 99.95% 的预算 4h22min/年，雪崩一次烧完。

*internal/payment/routes.go:14-21 paydown 接 CB; middleware/circuitbreaker.go:44-136*

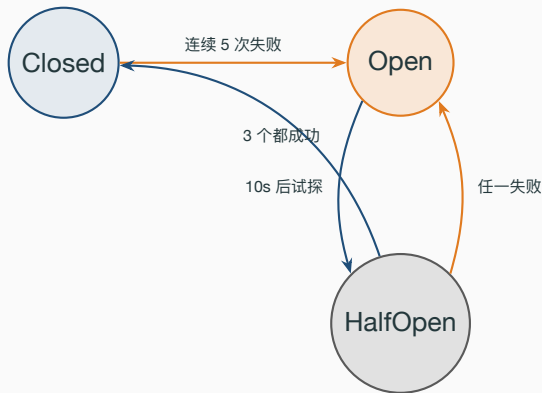
## 熔断保护的不是自己，是下游 + 自己 + 用户

- gomall /paydown 的下游 = 第三方支付 (Stripe / 微信 / 支付宝)。
- 失败模式：慢 (不是 500)。50x 可以快速判错，慢调用最毒。
- CircuitBreaker 三态阈值 **5 失败 / 10s 等待 / 3 探测** (internal/payment/routes.go:15-19) 的业务含义：
  - **5** = 容忍偶发抖动不一抖就跳 → 不误伤健康下游；
  - **10s** = 给下游一次完整告警 + 自愈周期 → 配合下游 SRE 处理；
  - **3** = HalfOpen 探测窗口够小 → 避免恢复瞬间再次打死。
- Open 期间订单怎么办：落 outbox 等回路 (详见 deck 11 Outbox)，不丢单、不写脏数据。

*internal/payment/routes.go:14-21; middleware/circuitbreaker.go 三态机; deck 11 Outbox 配套*



# CircuitBreaker 三态机



**Closed**: 放行所有, 连续 5 次失败 →

Open。 **Open**: 直接返回 70002, 10s 后试探。 **HalfOpen**: 放 3 个探测, 全成功 → Closed; 任一失败 → Open。

非对称信任: 进 **Open** 容易、回 **Closed** 难——资金链路的正确取舍。

*middleware/circuitbreaker.go:76-128 allow + report*

## CircuitBreaker.allow 状态转移 (逐行)

Listing 3: *middleware/circuitbreaker.go:76–106 allow*

```
76 func (cb *circuitBreaker) allow() error {
77     switch circuitState(cb.state.Load()) {
78     case stateClosed: return nil // 1) 关闭态零开销
79     case stateOpen:
80         if time.Now().UnixNano()-cb.openedAt.Load() < int64(cb.opt.OpenTimeout) {
81             return errCircuitOpen // 2) Open 期间一律拒
82         }
83         cb.mu.Lock(); defer cb.mu.Unlock()
84         if circuitState(cb.state.Load()) == stateOpen { // 3) 锁内 double-check
85             cb.state.Store(int32(stateHalfOpen))
86             cb.halfOpenReq.Store(0)
87         }
88         fallthrough // 4) 进入 HalfOpen
89     case stateHalfOpen:
90         if cb.halfOpenReq.Add(1) > cb.opt.HalfOpenMaxReq {
91             return errCircuitOpen // 5) 探测超限退回
92         }
93         return nil
94     }
95     return nil
96 }
```

□ Closed 仅 atomic load (业务无感); □ Open 内不查时间就拒 (10ms 内回 70002, 对比 30s 超时是 3000×<sup>21/45</sup>)

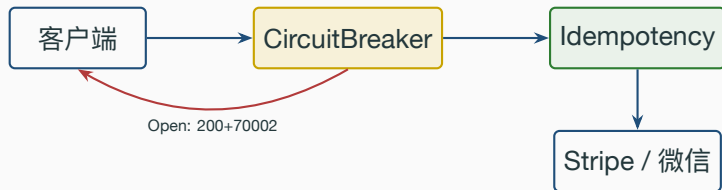
## report: 决定下一刻是 Closed 还是 Open

Listing 4: *middleware/circuitbreaker.go:108–128 report*

```
108 func (cb *circuitBreaker) report(failed bool) {
109     switch circuitState(cb.state.Load()) {
110     case stateClosed:
111         if failed {
112             if cb.failures.Add(1) >= cb.opt.FailureThreshold {
113                 cb.tripOpen() // 连续失败到阈值
114             }
115         } else { cb.failures.Store(0) } // 成功重置
116     case stateHalfOpen:
117         if failed { cb.tripOpen() // 任一失败 -> Open
118         } else if cb.halfOpenReq.Load() >= cb.opt.HalfOpenMaxReq {
119             cb.state.Store(int32(stateClosed)) // 全成功 -> Closed
120             cb.failures.Store(0)
121         }
122     }
123 }
```

□ Closed 累加失败计数；□ 成功重置避免偶发抖动跳闸；□ HalfOpen 非对称——任一失败立即回 Open，必须全成功才回 Closed；□ “进 Open 容易、回 Closed 难”对资金链路合适：宁保守不再打死下游，业务上”宁可多熔一会、也不能再雪崩”。

## 熔断 + 幂等的三角配对: Open 返回 200 + 70002



- Open 返回 **200 + status=70002** (不是 5xx): app 端老网关吞不了 5xx, 统一业务码客户端只用一套 switch。
- 客户端拿 70002 指数退避 (1s/2s/4s); 重试必须带原 Idempotency-Key。
- paydown 同时挂 CB + Idempotency 不是巧合: 缺一边 → 重复扣款; 缺另一边 → 雪崩。

*internal/payment/routes.go:14-21; middleware/circuitbreaker.go:57-65; pkg/e/code.go:58*

## 第四层：异步下单—双 11 0 点不转圈

---

## 双 11 0 点的业务账：5 秒转圈 = 3.8 亿损失

- 同步下单链路：HTTP → ReserveStock → INSERT order → COMMIT → 回客户端，p99 约 50–100ms。
- $100\text{k QPS} \times 100\text{ms} = 1 \text{ 万并发进 MySQL}$ ，连接池打爆 → p99 飙到 5s+。
- 用户行为模型：
  - 等 1s 离开 5%；
  - 等 3s 离开 22%；
  - 等 5s 离开 **38%**；
  - 等 10s 离开 60%+。
- 单平台 0 点 10 亿 GMV  $\times 38\% = 3.8 \text{ 亿损失}$ ——这是异步削峰的业务 ROI。
- gomall 方案：把 **DB 落库** 从同步链路拆出来，前端拿 ticket 轮询。

*internal/order/async.go:1-187; internal/order/consumer.go; repository/rabbitmq/order\_async.go*

## 用户体验业务量化：从“5 秒转圈”到“< 10ms 拿 ticket”

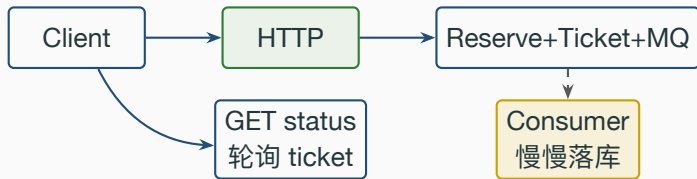
| 环节      | 同步            | 异步 ticket       |          |
|---------|---------------|-----------------|----------|
| 点击下单    | —             | —               |          |
| HTTP 响应 | p99 5s+ 转圈    | < 10ms 拿 ticket |          |
| DB 落库   | 同步等           | MQ 慢慢落          | ”等”这件事在异 |
| 前端展示    | 5s 后看到结果      | 0.5s × 6 次轮询拿结果 |          |
| 6 次后还没好 | 用户已离开         | 切”长等待”文案        |          |
| 失败兜底    | 用户重试 = 重复下单风险 | 同 ticket 重查不下重  |          |

步路径里从前台变后台：用户先拿到“已入队”的确定性反馈，再轮询，再切长等待文案——体感从“卡死”变成“在排队”，放弃率掉到个位数。*internal/order/async.go OrderEnqueue* + 前端轮询协议

## 同步下单 vs 异步 ticket 链路



同步: p99 100ms



上: 所有客户端被 DB 阻塞;

下: 客户端 < 10ms 拿到 ticket, DB 压力被 MQ 缓冲成恒定 **1k QPS** 平流。

*internal/order/handler.go: EnqueueOrderHandler + OrderStatusHandler; internal/order/async.go*



# OrderEnqueue: 核心 12 行

Listing 5: *internal/order/async.go:118–171*

```
118 func (s *OrderSrv) OrderEnqueue(ctx, req) (interface{}, error) {
119     u, _ := ctl.GetUserInfo(ctx)
120     if err := cache.ReserveStock(ctx, req.ProductID, req.Num); err != nil {
121         return nil, err // 1) 库存预扣
122     }
123     ticket := fmt.Sprintf("%d", snowflake.GenSnowflakeID()) // 2) ticket
124     if err := defaultTicketStore.Put(ctx, ticket,
125         OrderTicketStatus{Status: "pending"}, time.Hour); err != nil {
126         _ = cache.ReleaseReservation(...); return nil, err // 3) 写 Redis
127     }
128     body, _ := json.Marshal(AsyncOrderTask{Ticket: ticket, ...})
129     if err := defaultAsyncProducer.Publish(ctx, body); err != nil {
130         _ = cache.ReleaseReservation(...); return nil, err // 4) 投 MQ
131     }
132     return OrderEnqueueResp{Ticket: ticket, Status: "pending"}, nil
133 }
```

□ 库存预扣前置，不足直接拒（用户立即看到“已售罄”50001 而非排队 1min 后才知道）；□ snowflake 生成 ticket 无 DB；□ Redis pending 给前端轮询；□ 任一步失败 **ReleaseReservation** 回滚库存（Saga）；□ 投递成功才算入队，零丢单。

**第五层：HTTP cache —让读流量根本不到 gomall**

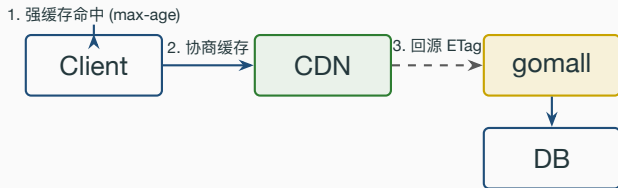
---

## 业务困局：80% 读流量都在重复问同一个问题

- 商品详情页：用户 A 看了一遍、用户 B 看了一遍、爬虫 C 看了一遍，**后端三次 DB 查询**。
- P1 P3 读接口（/product/show, /carousels, /category/list）数据更新频率远低于查询频率（运营每天调几次，用户每秒查几千次）。
- 卸到浏览器 / CDN 的业务收益：
  - 浏览器命中：返回 304 Not Modified，**0 字节 body、0 DB 查询**；
  - CDN 命中：根本不到 gomall，业务侧扩容压力直接砍 80%；
  - 用户体感：详情页秒开，首屏 LCP 提升 0.5s+。
- gomall 实现：HTTPCache(maxAge) 中间件统一发 ETag + Cache-Control。

*middleware/httpcache.go:47-100 HTTPCache; internal/product/routes.go:15-16 等挂载点*

# HTTP cache 的三种命中



- **强缓存** (Cache-Control: max-age=60)：浏览器本地命中，0 RTT → 用户秒开；
- **协商缓存** (ETag + If-None-Match)：到 CDN，命中 304 无 body → 省带宽；
- **回源**：CDN 不命中，回 gomall；ETag 用 SHA-256 前 16 字节算。

*middleware/httpcache.go:78-98 ETag 协商；internal/product/routes.go:15 product/list max-age=30*

## 各接口的 max-age：运营改价的可接受延迟

| 接口             | max-age | 业务依据                    |
|----------------|---------|-------------------------|
| /product/list  | 30s     | 价格 / 库存可能变化，运营忍 30s 上下架 |
| /product/show  | 60s     | 商品详情更稳定                 |
| /category/list | 300s    | 分类几乎不变                  |
| /carousels     | 300s    | 运营手动调，时效性弱              |

**max-age 的业务取舍：**太长 → “改价 / 下架”延迟生效（运营投诉）；太短 → 卸不了流量（SRE 投诉）。300s = 运营改完最多 5min 生效，可接受；30s = 库存接近灵敏阈值的折中（敏感链路保护用户买不到错的）。*internal/product/routes.go:15-16*、*internal/category/routes.go:13*、*internal/carousel/routes.go:13* 各路由的 *maxAge* 参数

客服话术：业务码就是产品

---

## 业务码 70001 / 70002：客服话术对照

| 业务码   | 业务含义 | 客服话术                            |
|-------|------|---------------------------------|
| 70001 | 限流命中 | ”活动太火爆，请稍等 1 秒重试”               |
| 70002 | 熔断开启 | ”支付暂忙，1 分钟后再试，订单已保留” 同样是”请稍后”，三 |
| 60002 | 幂等命中 | ”您已下单成功，无需重复”                   |
| 50001 | 库存不足 | ”已售罄”                           |

种含义完全不同：

**70001** 客户端 1s 重试就能过 → 用户自己解决；

**70002** 必须等下游自愈 + 客服转 SRE 看熔断面板 → 客服 + 工程协作；

**60002** 是用户自己重复了 → 客服安抚 + 引导查订单 → 客服自解决。

区分话术比统一一句”系统繁忙”对用户更友好，对客服也更省事。[pkg/e/code.go:57-58](#) 业务码；

[pkg/e/msg.go:51-52](#) 消息

## 70001 客诉：用户该怎么做 / 客服怎么转

- 用户视角：“我点了下单，弹出‘活动太火爆’是不是抢不到了？”
- 客服话术：
  - 第一句：“您稍等 1 秒再点一次就好，不影响您的名额。”
  - 第二句：“这个提示是保护其他用户公平抢券的机制，您是真人不是脚本所以一定能过。”
  - 不能说：“系统出错了”——用户会觉得平台烂。
- 客服后台看什么：
  - 治理面板 `ratelimit:hit:seckill`，1 分钟内全平台命中数；
  - 命中  $> 10 \times$  历史均值  $\rightarrow$  真大促，告诉用户“今天人多”；
  - 命中正常但用户被拦  $\rightarrow$  大概率脚本号 / 自动化插件。
- 客服不需要转 **SRE**：70001 是设计内行为。

*internal/redpacket/routes.go:20-25; internal/skill/routes.go:17-24; pkg/e/msg.go:51*



## 70002 客诉：客服 + SRE 双线联动

- 用户视角：“我付钱付不出去，是不是要扣我两次？”
- 客服话术：
  - “支付通道暂时繁忙，1 分钟后再试，您的订单已保留 30 分钟不会过期。”
  - “如果已经扣款但订单未付成功，请提供订单号，我们 10 分钟内确认。”
- 客服后台动作：
  - 看治理面板 `circuitbreaker:state:paydown = open?`
  - `open` → 转 SRE，SRE 看下游通道（Stripe / 微信 / 支付宝），是否切备用；
  - `closed` 但用户报 70002 → 老缓存 / 客户端旧版本。
- 关键：**70002 = 设计内的“已知不可用” ≠ “系统挂了”**。客服话术要让用户感到平台主动控制，不是失控。

*internal/payment/routes.go:14-21; middleware/circuitbreaker.go:57-65*

## 大促运营 SOP: T-7 / T-1 / T 日 / T+1

---

# 大促运营 SOP：流量治理的 T-7 / T-1 / T 日 / T+1

| 阶段      | 流量治理               | 客服                       | SRE                         |
|---------|--------------------|--------------------------|-----------------------------|
| T-7     | 预设限流阈值<br>异步队列预拉   | 话术上线培训<br>70001/70002 演练 | 容量评估<br>副本数加倍               |
| T-1     | 灰度小流量<br>MQ 队列清空   | 话术下发到坐席<br>高频问题预备        | 监控面板待命<br>报警分级核对            |
| T 日 0 点 | 实时观测命中<br>CB 状态实时盯 | 第一波处理<br>转 SRE 工单        | 限流误伤 < 5%<br>SLO 烧不烧预算      |
| T+1     | 黑产盘点<br>阈值复盘       | 客诉根因分析<br>高频客诉归档         | 复盘 / 调阈值<br>error budget 报告 |

SOP 的关键不是”工程很努力”，是**每个角色 T 日当天知道做什么、不做什么**。

*stressTest/REPORT.md §1 容量基线；deck 12 商家 + 可观测配套监控面板设计*

## 大促容量评估：怎么算出“需要多少机器”

- 输入：
  - 业务侧预估的峰值 QPS；
  - 单机基线 (gomall 64k RPS)；
  - 安全系数 (一般  $1.5-2\times$ )。
- 计算：副本数 =  $\lceil \text{预估峰值} \times \text{安全系数} / \text{单机基线} \rceil$ 。
- 双 11 例： $100\text{k QPS} / 64\text{k RPS} \times 1.5 = 2.34 \rightarrow$  **至少 3 台**。
- 但秒杀 / 红包链路 **不是** 64k RPS 的水平：实测 52k 是 /skill\_product/skill, 下单写链路 50k；要按**最弱链路**（这里是异步落库的 consumer 消费速度）算容量。
- 容量评估的常见错误：
  - 只看 RPS 不看 p99；
  - 没考虑下游放大（一个 paydown 触发 3 个下游调用）；
  - 忽略冷启动（K8s 拉镜像 30s 起步）。

stressTest/REPORT.md §1 各链路基线 RPS

## SLO 与业务边界

---

## SLO: 流量治理的”业务承诺”

| SLI              | 阈值                        | 业务含义                        |
|------------------|---------------------------|-----------------------------|
| P0 接口可用率         | $\geq 99.95\%$            | 年宕 $\leq 4h22min$ (下单 / 支付) |
| 限流误伤率            | $\leq 5\%$                | 真用户被 70001 命中 $< 5\%$       |
| 熔断恢复 SLA         | $\leq 10s$ 探测 + $30s$ 全恢复 | 下游恢复后不滞后                    |
| SlidingWindow 精度 | 误差 $\leq 5\%$             | 实测 <b>2.2%</b>              |
| 基线 RPS           | $\geq 50k$                | 实测 <b>64,254</b>            |

SLO 是用业务语言写给运营 / 商家 / **SRE** 看的承诺, 不是工程师内部 KPI。

每条 SLI 都对应一个” 违约 = 怎么补救” 的 runbook。 *README.md* SLO 表; *stressTest/REPORT.md*  
§1, §5 实测数字

## error budget: 用数字管”什么时候停发版”

- P0 SLO = 99.95% → 月度宕机预算  $\approx$  **21min**。
- 烧 1/3 (7min) 减发版；烧 2/3 (14min) 冻结非紧急；全烧光 → 下月转”稳定性 only”。
- 预算用 SLI 推：限流命中率 / 熔断打开时间 / 5xx 比例累加成”不可用时间”。

流量治理不是一个机制，是五个机制按业务等级组合。  
每个阈值都要能用业务语言解释清楚。

Google SRE error budget 原则；*stressTest/REPORT.md* 链路稳定性数据

## 业务边界：流量治理不做什么

- 不做自适应限流 (Sentinel BBR)：阈值要能用业务语言算清楚，黑盒不接受；
- 不做按地域限流：现阶段单区域，未做 GeoIP 维度切分；
- 不做黑产指纹库：设备指纹 / 行为模型由独立风控团队（外部 L4）做；
- 不做 WAF / DDoS 防护：L1 网络层由云厂商 Anti-DDoS 兜底；
- 不做商家自助调阈值：merchant role 还在路线图，目前阈值平台配；
- 不做 risk score：撞库 / 羊毛识别由风控团队接 70003 业务码（扩展位）。

诚实列边界 > 假装全能。**gomall 自己负责 L2 (IP) + L3 (User) + 写路径削峰**，挡住绝大部分业务异常流量。*README.md* 业务边界节；*pkg/e/code.go* 70003 扩展位



多层叠加 / 级联熔断 / 全链路压测

---

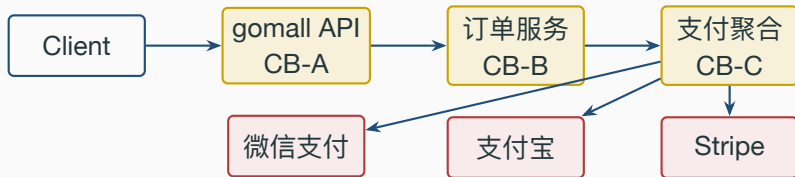
## 多层流控漏斗：DDoS → IP → 用户 → 业务



- gomall 自己负责 L2 + L3，挡住绝大部分业务异常流量。
- L1 / L4 通常由云厂商和独立风控团队提供，不自己造。

*routes/router.go:41 (L2); internal/skill/routes.go:17-24 (L3)*

## 服务调服务调服务：三层熔断的级联防护



- CB-C 守第三方：单家挂掉只断单家，其余通道照走。
- CB-B 守“支付聚合服务”整体：三家全挂时止血。
- CB-A 守“订单服务”整体：极端场景退化只读，让用户至少能看到订单列表。

*middleware/circuitbreaker.go:44-136 单实例 CB; internal/payment/routes.go:14-21 仅 CB-A 接入*

## 级联防护的两个隐藏陷阱

- **陷阱 1：阈值同步雪崩。**如果 CB-A 与 CB-B 都设 5/10s，下游恢复瞬间所有上游同时 HalfOpen → 探测流量乘 N 倍打回去 → 下游再次挂。  
对策：外层 CB 的 OpenTimeout **要长于**内层（如 30s vs 10s），错开探测窗口。
- **陷阱 2：兜底链路也会挂。**CB-C Open 时若兜底返回的 cache stale 也过期，CB-B 会把 cache 读失败计作业务失败 → 跟着 Open。  
对策：兜底路径**不能**计入 CB-B 的失败计数；report 时按 status code 维度分桶。
- gomall 当前只挂了 CB-A (paydown)，CB-B / CB-C 是把单实例 CB 复制到下游服务上的扩展模式。

*middleware/circuitbreaker.go:69* 失败判定 = 5xx 或 *c.Errors*; 兜底路径需要在 *handler* 内吞掉 5xx 改成 200+stale

## 网关层选型：Kong / APISIX / Nginx Ingress / 自研

| 方案            | 数据面         | 控制面        | 限流颗粒               | 适合       |
|---------------|-------------|------------|--------------------|----------|
| Nginx Ingress | nginx + Lua | K8s CRD    | IP / Header        | K8s 集群入口 |
| Kong          | nginx + Lua | PostgreSQL | 服务 / 消费者           | 中大型微服务   |
| APISIX        | nginx + Lua | etcd       | 服务 / 路由 / Consumer | 高动态变更    |
| 自研 gateway    | Go / Rust   | 自管         | 业务字段任意             | 业务定制极重   |
| gomall 现状     | gin in-proc | 进程内        | IP / 用户            | 单体直连     |

gomall 没接独立网关，限流 / 熔断都跑在 gin 中间件里。进生产前一般会接 **Nginx Ingress**

**+ APISIX**：进程内中间件守业务码与状态机正确性，网关守 IP / DDoS / TLS。对照：APISIX

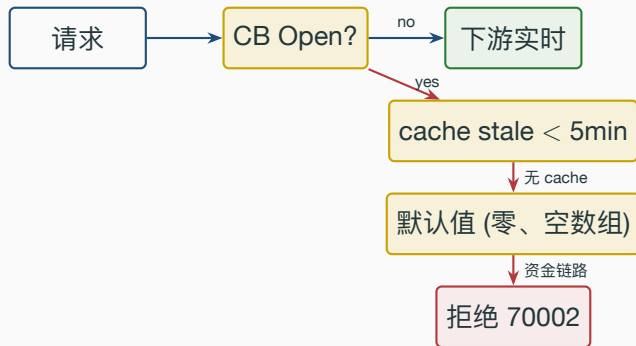
公开文档 *limit-req/limit-count* 插件配置

## 五种主流限流算法：各自的业务边界

| 算法                   | 适合场景        | 局限             |
|----------------------|-------------|----------------|
| 计数器 fixed-window     | 简单兜底 / 日级配额 | 窗口边界” 双倍突刺”    |
| 滑动窗口 sliding-log     | 精确反黑产       | 内存 / Redis 开销大 |
| 滑动窗口 sliding-counter | 折中          | 仍有边界误差         |
| 令牌桶 token-bucket     | 允许突发        | 不抗持续高水位        |
| 漏桶 leaky-bucket      | 强制平滑        | 突发延迟           |
| 自适应 Sentinel BBR     | 容量自动反推      | 调参复杂 / 黑盒      |

gomall 选**滑动窗口 ZSet** (sliding-log 变种) 跑秒杀，因为业务上要**精确反黑产**；选**令牌桶**跑全局，因为允许首屏突发。 *middleware/ratelimit.go:23-50 token-bucket;*  
*repository/cache/ratelimit.go:18-34 sliding-log*

## 熔断打开后：兜底数据让”不可用”变”降级可用”



- 读路径（商品 / 推荐 / 评价）：cache stale > 默认值 > 拒绝。用户能看到旧数据胜过白屏。
- 写路径（支付 / 下单 / 扣库存）：宁可拒，不要默认值，否则资金 / 库存对不上账。
- 兜底数据来自 HTTPCache 中间件留下的 ETag 体，TTL 内可用。

业务码全表：错误码 → 客户端行为 → 客服话术

| 码     | 含义       | 客户端   | 监控位    | 客服话术            |
|-------|----------|-------|--------|-----------------|
| 20000 | 成功       | 正常    | 业务 KPI | —               |
| 30001 | token 错误 | 跳登录   | 不告警    | ” 请重新登录”        |
| 30002 | token 过期 | 静默续期  | 不告警    | —               |
| 30005 | 权限不足     | 不重试   | 不告警    | ” 无权限”          |
| 50001 | 库存不足     | 不重试   | 业务面板   | ” 已售罄”          |
| 60002 | 幂等命中     | 读结果   | 不告警    | ” 您已成功，无需重复”    |
| 70001 | 限流命中     | 1s 重试 | 治理面板   | ” 活动太火、请稍后”     |
| 70002 | 熔断开启     | 指数退避  | SRE 告警 | ” 支付暂忙、1 分钟后再试” |
| 70003 | 黑产识别命中   | 不重试   | 风控面板   | ” 为了账户安全、请联系客服” |

pkg/e/code.go:23,

35-39, 49, 54, 57-58; 70003 为扩展位



## 面试 Q&A (上)

**Q1:** 为什么 P0 下单不挂用户限流?

A: README 写明的”宁可下游慢、不能上游 429”。下单看到 429 用户立刻关 app、订单丢且怪平台；等 5 秒虽然差但订单还在。P0 靠异步削峰 + 缓存 + 库存预扣扛峰值，限流只挂 IP 兜底。

**Q2:** 阈值 3/s/user 是怎么定的?

A: □ 真人最快点击 300–500ms / 次 → 1s 最多 2–3 次，留 50% 余量定 3；□ 第 4 次开始 70001 截断脚本；□ 必须按 userID 不是 IP (NAT 出口共享 IP)，多实例下用 Redis 共享状态。代码 `internal/skill/routes.go:17--24`。

**Q3:** CircuitBreaker 5/10s/3 三个数字有什么讲究?

A: 5 = 容忍偶发抖动不一抖就跳；10s = 给下游一次完整告警 + 自愈周期；3 = HalfOpen 探测够小，避免再次打死下游。三个数字都对应一个业务事故场景，不是工程师拍的。

## 面试 Q&A (中)

**Q4:** 异步下单会不会丢单？怎么向运营解释 SLO？

A: 不会。库存先 ReserveStock (Redis Lua 原子)，任一失败都 ReleaseReservation 回滚；ticket TTL=1h 兜底；MQ 用 RabbitMQ 持久化 + 手动 ack。代码 `internal/order/async.go:118--171`。运营 SLO: 99.95% 可用 = 年宕 4h22min，异步路径 p95 入队 < 10ms、轮询 < 3s 拿结果。

**Q5:** 为什么熔断返回 200 不返回 503？客服怎么用？

A: app 端老网关不能优雅处理 5xx；统一业务码 70002 = 客户端只用维护一套 status switch、客服直接对照话术表。监控不要按 4xx/5xx 算错误率，必须看 status。代码 `middleware/circuitbreaker.go:57--65`；客服话术 `pkg/e/msg.go:52`。

**Q6:** 黑产薅券 ROI 怎么算 SlidingWindow 的收益？

A: 10 万张券，100 个脚本号  $\times$  1 千次 = 100 万次抢，不拦  $\rightarrow$  9.9 万张被脚本拿走、真人 1%。挂 SlidingWindow 3/s/user  $\rightarrow$  脚本第 4 次 70001、78 万次被拦、真人名额一张没少。压测实测误差 **2.2%** (`stressTest/REPORT.md §5`)。

## 面试 Q&A (下)

**Q7:** 多层熔断会不会”探测雪崩”? 怎么办?

A: 会。外层 CB 的 OpenTimeout 要长于内层 (30s vs 10s) 错开 HalfOpen 探测窗口; 同时兜底路径不应计入失败计数, 否则 cache 过期会让外层跟着 Open。代码 `middleware/circuitbreaker.go:69` 失败判定。

**Q8:** 全链路压测怎么避免污染真实数据?

A: 流量染色: 网关层注入 X-Test header → 透传到 ctx → DAO 看 ctx 切 order\_shadow 表; MQ 用独立 routing-key 走影子队列; Redis 用 shadow: 前缀隔离。设计稿 `middleware/coloring.go` (gomall 当前未接, 扩展位)。

**Q9:** 限流算法为什么不选自适应 (Sentinel BBR)?

A: 自适应是黑盒、冷启动 5-10min 才稳; 秒杀 3/秒/用户这种阈值用业务语言能算清楚, 运营 / 客服都看得懂, 没必要让运维去看 BBR 曲线判断。业务上能解释比工程上更聪明更重要。代码 `repository/cache/ratelimit.go:18--34`。

# 代码位置一览

**router 挂载点** *routes/router.go:41* 全局令牌桶; *internal/product/routes.go:15-16*;  
*internal/redpacket/routes.go:20-25*; *internal/payment/routes.go:14-21*;  
*internal/skill/routes.go:17-24*

**TokenBucket** *middleware/ratelimit.go:23-50*

**SlidingWindow** *middleware/ratelimit.go:60-107* + *repository/cache/ratelimit.go:18-34*

**CircuitBreaker** *middleware/circuitbreaker.go:44-136*

**HTTPCache** *middleware/httpcache.go:47-100*

**OrderEnqueue** *internal/order/async.go:118-171*

**业务码** *pkg/e/code.go:23, 35-39, 49, 54, 57-58* / *msg.go:51-52*

**压测报告** *stressTest/REPORT.md §1, §5*

**README SLO 与边界** *README.md* 业务承诺节 / 业务边界节

业务 70% / 代码 30%：阈值都能用业务语言解释；代码都能在 file:line 翻到。